

```
-----  
-----  
-- Engineer: Olasubomi Borishade  
--  
-- Create Date: 05/30/2026 06:13:16 PM  
-- Design Name: Processing Element  
-- Module Name: Processing_Element - Behavioral  
-- Project Name: Parameterized Matrix Multiply Engine  
  
-- Description: Wraps a MAC unit and forwards Input_A rightward and  
Input_B  
--                downward to neighboring processing elements,  
--                with one register stage per direction.  
  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
-----
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.NUMERIC_STD.ALL;  
  
-- Uncomment the following library declaration if instantiating  
-- any Xilinx leaf cells in this code.  
--library UNISIM;  
--use UNISIM.VComponents.all;  
  
entity Processing_Element is  
  
    Generic (  
        Input_Width          : INTEGER := 8;  
        Accumulator_Width    : INTEGER := 32  
    );
```

```

Port      (    Input_A      : in  SIGNED (Input_Width - 1
downto 0));

          Input_B          : in  SIGNED (Input_Width - 1
downto 0);

          Clock             : in  STD_LOGIC;
          Reset             : in  STD_LOGIC;
          Enable            : in  STD_LOGIC;
          Clear_Accumulator : in  STD_LOGIC;
          Output_A          : out SIGNED (Input_Width - 1
downto 0);

          Output_B          : out SIGNED (Input_Width - 1
downto 0);

          Output            : out SIGNED (Accumulator_Width
- 1 downto 0)
        );

```

end Processing_Element;

architecture Behavioral of Processing_Element is

```

Signal A_Forward      : SIGNED (Input_Width - 1 downto 0) :=
(others => '0');

Signal B_Forward      : SIGNED (Input_Width - 1 downto 0) :=
(others => '0');

```

begin

```

MAC_Unit_Inst : entity work.MAC_Unit(Behavioral)
  Generic map (    Input_Width      => Input_Width,
                  Accumulator_Width => Accumulator_Width
                )

  Port map      (    Input_A      => Input_A,
                  Input_B      => Input_B,
                  Clock        => Clock,
                  Reset        => Reset,
                  Clear_Accumulator => Clear_Accumulator,
                  Enable       => Enable,

```

Output
);

=> Output

```
process(Clock)
begin
    if rising_edge(Clock) then
        if Reset = '1' then
            A_Forward      <= (others => '0');
            B_Forward      <= (others => '0');
        elsif Enable = '1' then
            A_Forward      <= Input_A;
            B_Forward      <= Input_B;
        end if;
    end if;
end process;
```

Output_A <= A_Forward;

Output_B <= B_Forward;

end Behavioral;